

Java OOP Study Notes: Constructors, this Keyword & Static Members

1. Object Creation: The `new` Keyword

The `new` keyword is used to create objects (instances) of a class. It allocates memory for the object in the **heap** and returns a reference to it.

Syntax

```
ClassName objectName = new ClassName();
```

What Happens When You Use `new` ?

1. Memory is allocated for the new object
2. Instance variables are initialized to default values
3. The constructor is called
4. A reference to the object is returned

Example

```
Student s1 = new Student(); // Creates a new Student object
```

2. Constructors

A **constructor** is a special method that is automatically called when an object is created. It initializes the object's state.

Key Characteristics

Feature	Description
Name	Must be same as class name
Return Type	No return type (not even void)
Called	Automatically when object is created
Purpose	Initialize object's instance variables

3. Default Constructor

A **default constructor** is a constructor with **no parameters**. If you don't write any constructor, Java provides one automatically.

Compiler-Provided Default Constructor

```
class Student {  
    String name;  
    int age;  
    // Java provides: Student() { } automatically  
}  
  
// Usage  
Student s = new Student(); // name = null, age = 0
```

User-Defined Default Constructor

```
class Student {
    String name;
    int age;

    // Default constructor (no parameters)
    Student() {
        name = "Unknown";
        age = 0;
        System.out.println("Default constructor called!");
    }
}

// Usage
Student s = new Student(); // name = "Unknown", age = 0
```

Note: If you define ANY constructor, Java will NOT provide the default one automatically.

4. Parameterized Constructor

A **parameterized constructor** accepts arguments to initialize an object with specific values.

Syntax

```
class ClassName {
    ClassName(datatype param1, datatype param2) {
        // initialization code
    }
}
```

Example

```
class Student {
    String name;
    int age;

    // Parameterized constructor
    Student(String studentName, int studentAge) {
        name = studentName;
        age = studentAge;
    }

    void display() {
        System.out.println("Name: " + name + ", Age: " + age);
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Aslam", 20);
        Student s2 = new Student("Rahim", 22);

        s1.display(); // Output: Name: Aslam, Age: 20
        s2.display(); // Output: Name: Rahim, Age: 22
    }
}
```

5. Constructor Overloading

Constructor overloading means having multiple constructors in the same class with **different parameter lists**.

Rules for Overloading

- Same constructor name (class name)

- Different number of parameters, OR
- Different types of parameters, OR
- Different order of parameter types

Example

```
class Student {
    String name;
    int age;
    String course;

    // Constructor 1: No parameters (default)
    Student() {
        name = "Unknown";
        age = 0;
        course = "Not Assigned";
    }

    // Constructor 2: One parameter
    Student(String name) {
        this.name = name;
        age = 0;
        course = "Not Assigned";
    }

    // Constructor 3: Two parameters
    Student(String name, int age) {
        this.name = name;
        this.age = age;
        course = "Not Assigned";
    }

    // Constructor 4: Three parameters
    Student(String name, int age, String course) {
        this.name = name;
        this.age = age;
        this.course = course;
    }

    void display() {
        System.out.println(name + " | " + age + " | " + course);
    }
}
```

```
// Usage
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();           // Calls Constructor 1
        Student s2 = new Student("Aslam");    // Calls Constructor 2
        Student s3 = new Student("Rahim", 21); // Calls Constructor 3
        Student s4 = new Student("Karim", 22, "CSE"); // Calls Constructor 4

        s1.display(); // Unknown | 0 | Not Assigned
        s2.display(); // Aslam | 0 | Not Assigned
        s3.display(); // Rahim | 21 | Not Assigned
        s4.display(); // Karim | 22 | CSE
    }
}
```

6. The `this` Keyword

The `this` keyword is a **reference to the current object**. It's used inside instance methods and constructors.

Use Cases of `this`

6.1 Distinguish Instance Variables from Parameters

When parameter names are same as instance variable names:

```

class Student {
    String name;
    int age;

    Student(String name, int age) {
        this.name = name;    // this.name = instance variable
        this.age = age;      // name = parameter
    }
}

```

6.2 Call Another Constructor (Constructor Chaining)

```

class Student {
    String name;
    int age;

    Student() {
        this("Unknown", 0); // Calls parameterized constructor
        System.out.println("Default constructor");
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
        System.out.println("Parameterized constructor");
    }
}

// Usage
Student s = new Student();
// Output:
// Parameterized constructor
// Default constructor

```

Important: `this()` must be the **first statement** in a constructor.

6.3 Pass Current Object as Argument

```
class Student {
    String name;

    Student(String name) {
        this.name = name;
    }

    void enroll(Course c) {
        c.addStudent(this); // Pass current Student object
    }
}
```

6.4 Return Current Object

```
class Builder {
    String value = "";

    Builder append(String s) {
        value += s;
        return this; // Return current object for method chaining
    }
}

// Usage: Method Chaining
Builder b = new Builder();
b.append("Hello").append(" ").append("World");
```

7. Static Members

Static members belong to the **class** rather than any specific object. They are shared among all instances.

7.1 Static Variables

```
class Student {
    String name;           // Instance variable (each object has its own)
    static int totalStudents; // Static variable (shared by all objects)

    Student(String name) {
        this.name = name;
        totalStudents++;    // Increment for each new student
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student("Aslam");
        Student s2 = new Student("Rahim");
        Student s3 = new Student("Karim");

        System.out.println(Student.totalStudents); // Output: 3
    }
}
```

7.2 Static Methods

Static methods can be called **without creating an object**.

```

class MathUtils {
    static int add(int a, int b) {
        return a + b;
    }

    static int square(int n) {
        return n * n;
    }
}

// Usage - No object needed!
int sum = MathUtils.add(5, 3);    // 8
int sq = MathUtils.square(4);    // 16

```

Rules for Static Methods

Can Access	Cannot Access
Static variables	Instance variables (directly)
Static methods	Instance methods (directly)
	<code>this</code> keyword

```

class Example {
    int x = 10;        // Instance variable
    static int y = 20; // Static variable

    static void show() {
        // System.out.println(x); // ERROR! Cannot access instance variable
        System.out.println(y);    // OK - static can access static
    }
}

```

7.3 Static Block

A static block runs **once** when the class is loaded, before any objects are created.

```
class Database {
    static String connectionString;

    // Static block - runs when class loads
    static {
        System.out.println("Loading database configuration...");
        connectionString = "jdbc:mysql://localhost:3306/mydb";
    }

    Database() {
        System.out.println("Creating database connection");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        System.out.println("Main started");
        Database db1 = new Database();
        Database db2 = new Database();
    }
}

// Output:
// Main started
// Loading database configuration...
// Creating database connection
// Creating database connection
```

8. Quick Comparison Table

Feature	Instance Members	Static Members
Belongs to	Object	Class
Memory	Created for each object	Created once for class
Access	Through object reference	Through class name
Can use <code>this</code>	Yes	No
Keyword	None	<code>static</code>

9. Complete Example

```
class BankAccount {
    // Instance variables
    private String accountHolder;
    private double balance;

    // Static variable
    private static int totalAccounts = 0;
    private static double interestRate = 0.05;

    // Static block
    static {
        System.out.println("BankAccount class loaded");
    }

    // Default constructor
    BankAccount() {
        this("Unknown", 0);
    }

    // Parameterized constructor
    BankAccount(String accountHolder, double balance) {
        this.accountHolder = accountHolder;
        this.balance = balance;
        totalAccounts++;
    }

    // Instance method
    void deposit(double amount) {
        this.balance += amount;
    }

    // Instance method using static variable
    void addInterest() {
        this.balance += this.balance * interestRate;
    }

    // Static method
    static int getTotalAccounts() {
```

```

        return totalAccounts;
    }

    // Static method to change interest rate
    static void setInterestRate(double rate) {
        interestRate = rate;
    }

    void display() {
        System.out.println(accountHolder + ": $" + balance);
    }
}

public class Main {
    public static void main(String[] args) {
        // Create accounts
        BankAccount acc1 = new BankAccount("Aslam", 1000);
        BankAccount acc2 = new BankAccount("Rahim", 2000);

        // Use static method
        System.out.println("Total Accounts: " + BankAccount.getTotalAccounts());

        // Add interest
        acc1.addInterest();
        acc2.addInterest();

        acc1.display(); // Aslam: $1050.0
        acc2.display(); // Rahim: $2100.0
    }
}

```

10. Key Points to Remember

1. **Constructors** initialize objects and have no return type
2. **Default constructor** has no parameters; Java provides one if you don't write any constructor

3. **Parameterized constructors** let you set initial values when creating objects
4. **Constructor overloading** = multiple constructors with different parameters
5. **this** refers to the current object; used to resolve naming conflicts
6. **this()** calls another constructor; must be first statement
7. **Static members** belong to the class, not individual objects
8. **Static methods** cannot access instance members directly
9. **Static blocks** execute once when the class is loaded

```
'; triggerDownload(docTitle.replace(/^[^a-zA-Z0-9\s-]/g, '') + '.html', html, 'text/html;charset=utf-8'); }  
function downloadPDF() { document.getElementById('downloadDropdown').classList.remove('show'); //  
Force light mode for PDF (dark backgrounds print poorly) const wasDark =  
document.documentElement.classList.contains('dark'); if (wasDark)  
document.documentElement.classList.remove('dark'); const element =  
document.getElementById('document-content'); const opt = { margin: [0.75, 0.75, 0.75, 0.75], filename:  
docTitle.replace(/^[^a-zA-Z0-9\s-]/g, '') + '.pdf', image: { type: 'jpeg', quality: 0.98 }, html2canvas: { scale:  
1.5, useCORS: true, backgroundColor: '#ffffff' }, jsPDF: { unit: 'in', format: 'letter', orientation: 'portrait' },  
pagebreak: { mode: ['css', 'legacy'] } }; html2pdf().set(opt).from(element).save().then(function() { if  
(wasDark) document.documentElement.classList.add('dark'); }); } /* Syntax highlighting and mermaid  
rendering */ (async function init() { const isDark = window.matchMedia('(prefers-color-scheme:  
dark)').matches; // Highlight code blocks (skip mermaid) document.querySelectorAll('pre  
code').forEach(function(block) { const lang = block.className.replace('language-', ''); if (lang !==  
'mermaid') { hljs.highlightElement(block); }); updateHljsTheme(isDark); // Render mermaid diagrams try {  
mermaid.initialize({ startOnLoad: false, theme: isDark ? 'dark' : 'default', securityLevel: 'loose' }); const  
mermaidBlocks = document.querySelectorAll('pre code.language-mermaid'); for (let i = 0; i <  
mermaidBlocks.length; i++) { const block = mermaidBlocks[i]; const pre = block.parentElement; const  
code = block.textContent; try { const { svg } = await mermaid.render('mermaid-' + i, code); const  
container = document.createElement('div'); container.className = 'mermaid-container';  
container.innerHTML = svg; pre.replaceWith(container); } catch (err) { console.warn('Mermaid render  
failed:', err); } } } catch (err) { console.warn('Mermaid init failed:', err); } })(); function  
updateHljsTheme(isDark) { document.getElementById('hljs-dark').disabled = !isDark;  
document.getElementById('hljs-light').disabled = isDark; } /* Dark mode sync from parent */  
window.addEventListener('message', function(e) { if (e.data && e.data.type === 'darkMode') { const  
isDark = e.data.dark; document.documentElement.classList.toggle('dark', isDark);
```



```
updateHljsTheme(isDark); // Re-init mermaid theme try { mermaid.initialize({ startOnLoad: false, theme:  
isDark ? 'dark' : 'default', securityLevel: 'loose' }); } catch (err) { } });
```